

To-Do List Manager Mikroservisi için Uçtan Uca Test ve Dağıtım Altyapısı

Ders: MTH2526-B25 Bulut Mimarilerinde Test Mühendisliği | Proje: To-Do List Manager
Ekip: MHD DIAA ALSEBAI (170423954) , Yusuf Eren ÇELEBİ (170424826) | Teslim: 1 Haziran 2026

Özet

Bu proje, **To-Do List Manager** adlı küçük bir görev yönetimi mikroservisi üzerinde uçtan uca test, dağıtım ve gözlemlenebilirlik altyapısı kurmayı hedefler. Uygulama alanı sade tutulmuştur: görev oluşturma, listeleme, tamamlama, silme, etiketleme ve göreve dosya eki ekleme. Asıl odak, bu servis için test piramidi, API otomasyonu, container çalıştırma, Kubernetes manifestleri, CI/CD, monitoring ve performans ölçümünü birlikte çalışır hale getirmektir.

Kullanılan ana teknolojiler: FastAPI, SQLAlchemy, SQLite/PostgreSQL, LocalStack S3, Pytest, Factory Boy, Postman/Newman, Playwright, Docker, Kubernetes/Minikube, Prometheus, Grafana ve k6. Bonus kapsamda Helm chart, ArgoCD GitOps ve OpenTelemetry/Jaeger tracing eklendi.

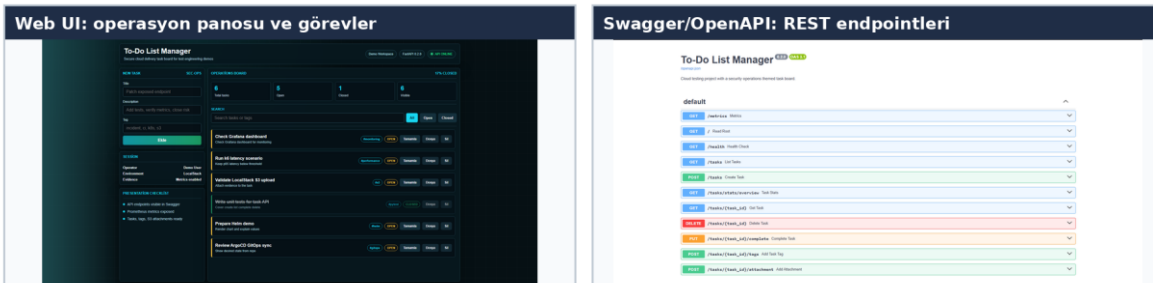
Sayısal Özet ve Rubric Uygunluğu

Kalem	Kanıt / Sonuç
REST API	Swagger üzerinde /metrics, /health, /tasks ve görev işlem endpointleri görünür.
Test sonucu	Pytest: 18 passed, 0 skipped, 0 warnings; toplam coverage: %75.18. Integration testler Docker açıkken PostgreSQL container üzerinde çalıştı.
Performans	k6 senaryosu: 10 VU, 30 sn; 300/300 successful; p95 latency: 40.47 ms, 500 ms eşiği altında.
Dağıtım	Docker Compose servisleri başladı; Minikube üzerinde 2 pod running ve NodePort servis hazır.
Gözlemlenebilirlik	Grafana 3 panel, Prometheus metrikleri, Jaeger trace kayıtları; ArgoCD Healthy/Synced.

1. Giriş

To-Do List Manager, demo ve test senaryoları için yeterli davranış yüzeyi sağlayan küçük bir uygulamadır. Kullanıcı arayüzünde operasyon panosu bulunur; görevler etiketlenebilir, tamamlanabilir, silinebilir ve dosya ekiyle ilişkilendirilebilir. Böylece API, veritabanı, obje depolama, UI ve monitoring bileşenleri tek bir uçtan uca akışta gösterilebilir.

Projenin amacı karmaşık bir ürün geliştirmekten çok, basit bir servis için profesyonel kalite hattı kurmaktır. Bu nedenle raporda uygulama özelliklerinden daha çok test çeşitliliği, otomasyon, dağıtılabilirlik ve gözlemlenebilirlik kararları açıklanmıştır.



Şekil 1. Uygulama arayüzü ve Swagger/OpenAPI endpoint görünümü.

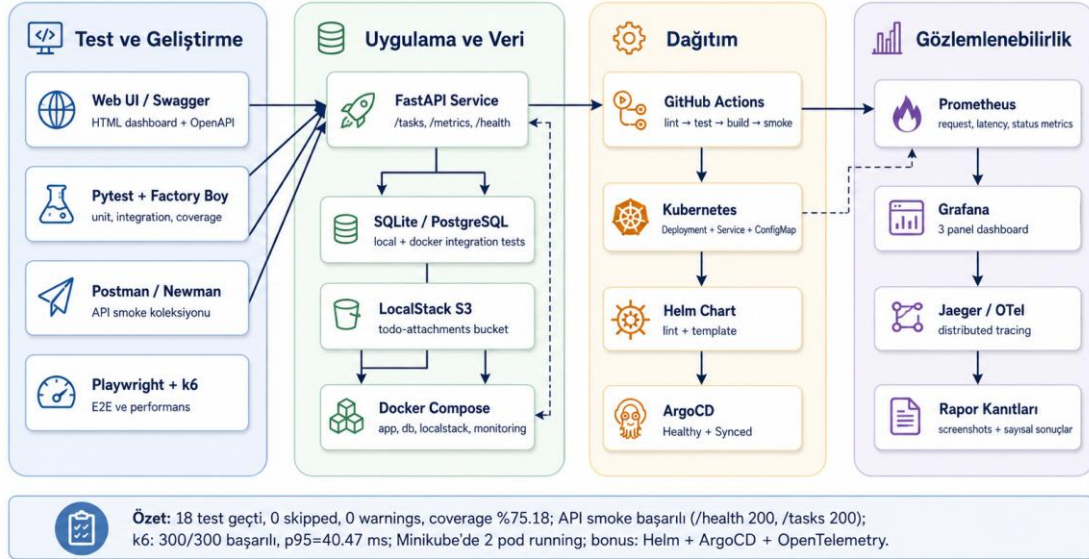
2. Mimari

Mimari üç ana eksenden oluşur: uygulama katmanı, veri/entegrasyon katmanı ve operasyon katmanı. Uygulama katmanında FastAPI servisleri, SQLAlchemy modelleri ve HTML/CSS/JS tabanlı tek ekran dashboard bulunur. Veri tarafında SQLite yerel geliştirme için, PostgreSQL Docker/Testcontainers senaryoları için, LocalStack S3 ise dosya

eki akışı için kullanılır. Operasyon tarafında Docker Compose, Kubernetes manifestleri, GitHub Actions, Prometheus/Grafana, Jaeger, k6 ve Playwright birlikte çalışır.

Docker Compose ortamında PostgreSQL, LocalStack, Prometheus, Grafana ve Jaeger servisleri birlikte ayağa kaldırılır. Kubernetes tarafında Deployment, Service ve ConfigMap dosyaları bulunur; Helm chart aynı manifestleri paketlenilebilir hale getirir. ArgoCD uygulama görünümü ise GitOps akışında kaynakların Healthy ve Synced durumda olduğunu gösterir.

To-Do List Manager - Uçtan Uca Test ve Dağıtım Mimarisi

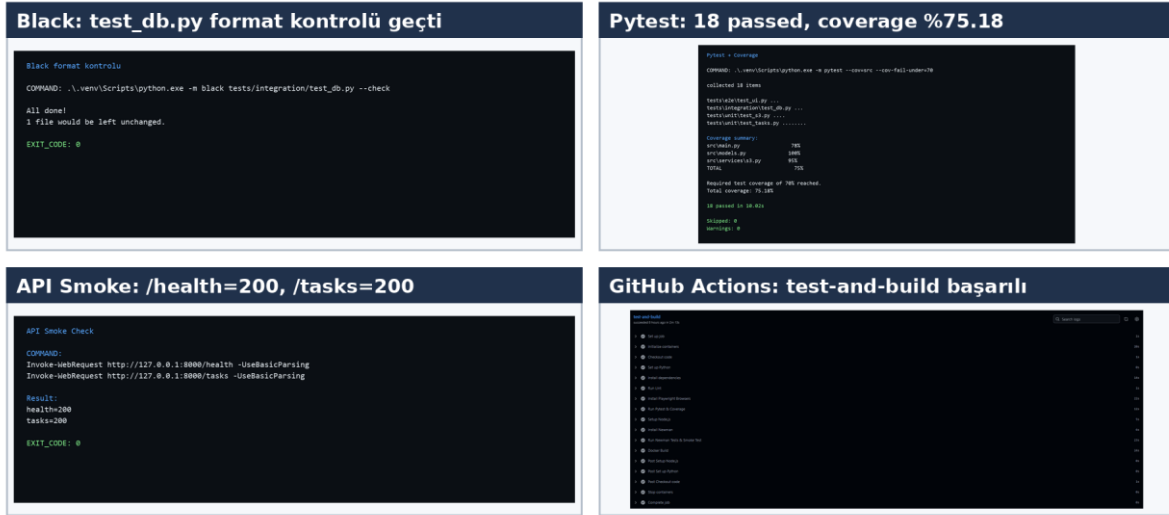


Şekil 2. Uygulama, test, dağıtım ve gözlemlenebilirlik bileşenlerinin mimari diyagramı.

3. Test Stratejisi

Test stratejisi test piramidine göre düzenlenmiştir. Hızlı geri bildirim için unit testler geniş tutulmuş, gerçekçi davranış için integration, API smoke, E2E ve performans testleri eklenmiştir. Coverage kapısı CI içinde --cov-fail-under=70 ile korunur. Güncel yerel kontrolde Docker açıkken 18 testin tamamı geçmiştir; skipped ve warnings değerleri 0, toplam coverage değeri ise %75.18 olarak ölçülmüştür. PostgreSQL integration testleri Docker SDK ile açılan postgres:16-alpine container üzerinde çalıştırılmıştır.

Katman	Araç	Amaç
Unit test	Pytest + Factory Boy	Endpoint davranışları, S3 yardımcı fonksiyonları ve hata durumları.
Integration	Testcontainers + PostgreSQL	Gerçek veritabanında create/update/delete akışları.
API smoke	Postman + Newman	Health, listeleme, oluşturma, tamamlama ve attachment upload.
E2E	Playwright	UI üzerinden görev ekleme, listeleme ve tamamlama senaryoları.
Performans	k6 senaryosu: 10 VU, 30 sn; 300/300 successful; p95 latency: 40.47 ms, 500 ms eşiği altında.	p95 latency hedefinin yük altında doğrulanması.

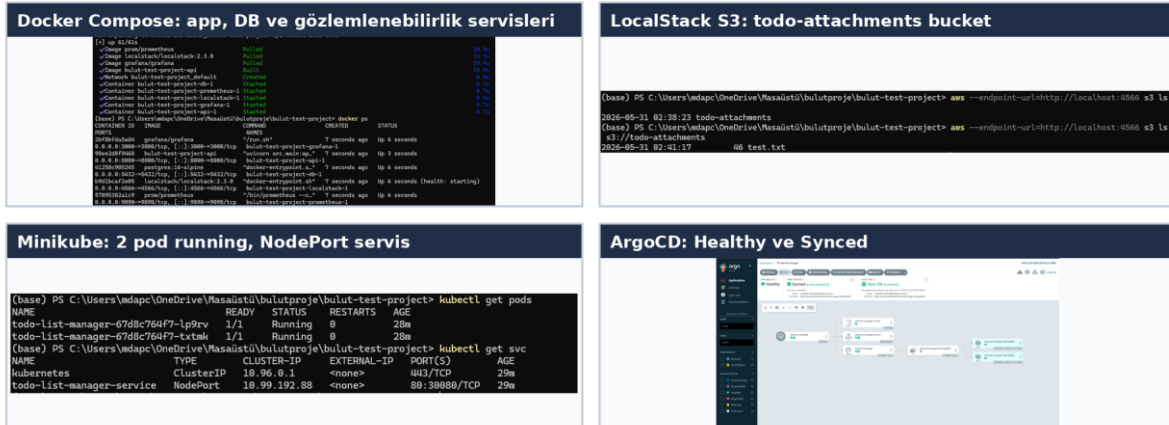


Şekil 3. Black format kontrolü, güncel Pytest coverage, API smoke ve GitHub Actions kanıtları.

4. Pipeline ve Dağıtım

GitHub Actions workflow tek akışta kalite kapılarını çalıştırır: checkout, Python kurulumu, bağımlılık kurulumu, Black lint kontrolü, Playwright browser kurulumu, Pytest coverage, Newman smoke testleri, Docker build ve Helm lint/template kontrolleri. Workflow ekran görüntüsünde test-and-build jobunun başarılı tamamlandığı ve adımların sırasıyla çalıştığı görülmektedir.

Dağıtım tarafında Docker Compose lokal demo ortamını tekrar üretilebilir hale getirir. Minikube üzerinde iki adet todo-list-manager podu running durumdadır ve servis NodePort olarak dışarı açılmıştır. Helm chart, Deployment/Service/ConfigMap kaynaklarını paketleyerek farklı ortamlara taşımayı kolaylaştırır; ArgoCD uygulaması da Git deposundaki istenen durumu cluster ile eşitlemektedir.

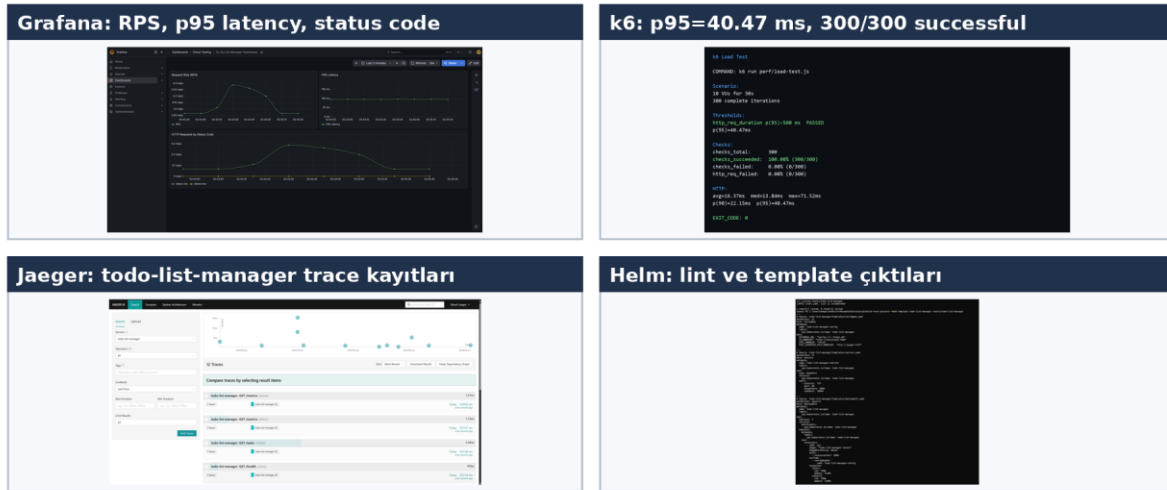


Şekil 4. Docker Compose, LocalStack S3, Minikube ve ArgoCD dağıtım kanıtları.

5. Performans ve Gözlemlenebilirlik

FastAPI uygulaması Prometheus exporter ile request rate, p95 latency ve HTTP status code dağılımı gibi metrikleri üretir. Grafana dashboard üzerinde bu metrikler üç anlamlı panelde izlenir. Bu yapı, canlı demo sırasında hem sistem yükünün hem de hata oranının hızla yorumlanmasını sağlar.

Performans testi k6 ile GET /tasks endpointine 10 sanal kullanıcı ve 30 saniyelik çalışma süresiyle uygulanmıştır. Toplam 300 check çalışmış, 300/300 successful sonucu alınmış, HTTP hata oranı %0 kalmış ve p95 latency 40.47 ms ölçülmüştür. Bu değer 500 ms eşliğinin altında kalarak performans hedefinin geçtiğini gösterir. OpenTelemetry/Jaeger entegrasyonu ise metriklerden farklı olarak tekil HTTP isteklerinin servis içindeki izini gösterir; Jaeger ekranında todo-list-manager servisine ait trace kayıtları görülmektedir.



Şekil 5. Grafana, k6 sonucu, Jaeger ve Helm çıktıları.

6. Sonuç ve Öğrenilenler

Proje, küçük bir mikroservis için uçtan uca kalite hattının nasıl kurulabileceğini göstermektedir. En önemli kazanım, testlerin tek seviyede bırakılmaması ve aynı sistemin unit, integration, API smoke, E2E ve performans perspektiflerinden doğrulanmasıdır. CI/CD pipeline ve container tabanlı çalışma biçimi, aynı sonuçların yerel makine, GitHub Actions ve Kubernetes ortamında tekrar üretilebilmesini sağlar.

- Test piramidi hızlı geri bildirim ile gerçekçi doğrulama arasında denge kurar.
- LocalStack, gerçek AWS hesabı gerektirmeden S3 davranışını test etmeyi sağlar.
- Docker Compose ve Minikube, demo ortamını tekrarlanabilir ve taşınabilir hale getirir.
- Grafana metrikleri sistem davranışını, Jaeger trace kayıtları ise tekil istek akışını görünür kılar.

7. İş Paylaşımı

Proje iki kişi olarak hazırlanmış ve sunulacaktır. Final sunumunda MHD problem, mimari ve test stratejisini; Yusuf Eren ise demo, deploy, monitoring ve performans akışını aktaracaktır. Aşağıdaki tablo, rapora eklenen özet iş paylaşımıdır; ayrıntılı sürüm repo içinde docs/work-distribution.md dosyasında tutulabilir.

Bölüm / Modül	Sorumlu	Açıklama
Mimari + rapor anlatımı	MHD	Problem, mimari kararlar, test stratejisi ve rapor akışı.
Demo + deploy	Yusuf Eren	CI/CD, Docker, Minikube, Helm/ArgoCD ve canlı demo akışı.
Monitoring + performans	Yusuf Eren	Grafana dashboard, k6 sonucu ve Jaeger trace yorumu.
Q&A hazırlığı	MHD + Yusuf Eren	Kod, coverage, Dockerfile, deploy ve rollback soruları.

8. Kaynaklar

- FastAPI Documentation
- Pytest Documentation
- LocalStack Documentation
- Grafana Documentation
- k6 Documentation
- Playwright Documentation
- Helm Documentation
- ArgoCD Documentation
- OpenTelemetry Documentation